DEFENSE REPORT

EagleMart Web Application

Prepared by: Team Cipher
Team Members:
Toby Holekamp
Chetanchal Saud
Benjamin Cooper
Christian Stuart
Binita Dhakal

Original Pentest Report by: Team Vortex
Date of Remediation: April 7, 2026 - April 11, 2026
Report Date: April 12, 2026
Course: CSC 489 Capstone Project

TABLE OF CONTENTS

1. Executive Summary 3
2. Vulnerabilities Addressed 3
3. Remediation Details 5
   3.1 Critical and High Severity Fixes 5
   3.2 Medium Severity Fixes 13
   3.3 Low Severity Fixes 15
4. Testing and Verification 16
   4.1 Testing Methodology 16
   4.2 Remediation Verification Results 16
   4.3 Functionality Testing 17
5. Lessons Learned 18
6. Conclusion 19
Appendix A. Files Modified 20

1. Executive Summary

This report presents the remediation work completed for EagleMart in response to the Milestone 2 penetration test conducted by Team Vortex. The purpose of this phase was to correct each confirmed vulnerability, verify that the documented exploits were no longer effective, and ensure that the application continued to function properly after the security fixes were applied.

All eight findings identified in the penetration test were addressed during remediation. The work was prioritized by severity. Critical vulnerabilities were resolved first because they created the greatest immediate risk to the application, particularly the SQL injection and cross-site scripting issues. High-severity findings were then corrected, followed by the remaining medium- and low-severity weaknesses. Each remediation step was validated using the same payloads and reproduction procedures documented in the original penetration test report.

At the conclusion of this phase, all reported vulnerabilities were remediated and retested successfully. EagleMart now uses parameterized database queries, secure frontend rendering practices, bcrypt password hashing, stronger file upload controls, restricted cross-origin access, and safer error handling. These changes significantly improve the application's security posture while preserving expected functionality.

The remediation materially improved the application's security posture by reducing opportunities for unauthorized access, limiting the impact of malicious input, strengthening the protection of sensitive data, and improving the security of exposed application workflows.

Total vulnerabilities found	**8**
Critical	2
High	3
Medium	2
Low	1
Overall remediation status	8 of 8

2. Vulnerabilities Addressed

Table 1 summarizes the vulnerabilities identified in the penetration test and the remediation status for each finding. All reported issues were reviewed, mapped to the affected code locations, corrected in the application, and then re-tested to confirm that the original exploit path was no longer effective.

#	Vulnerability	Severity	Location	Status
1	SQL Injection in Search	Critical	GET /api/search	Fixed
2	Cross-Site Scripting (Stored & Reflected XSS)	Critical	/listings, /search, /	Fixed
3	IDOR with Plaintext Password Exposure	High	GET /api/profile/<id>	Fixed
4	Default Credentials in HTML Source	High	/login (source)	Fixed
5	Unrestricted File Upload	High	POST /api/listings	Fixed
6	Username Enumeration	Medium	POST /api/login	Fixed
7	Missing CSRF Protection	Medium	All POST endpoints	Fixed
8	Verbose Error Messages	Low	GET /api/search	Fixed

Remediation Approach

The remediation effort was carried out in a structured order based on severity and potential impact. Critical and high-severity findings were addressed first because they created the most serious security risk, including unauthorized database access, browser-based code execution, exposure of sensitive credentials, and unsafe file handling. These issues had the greatest potential to affect confidentiality, integrity, and user trust, so they were treated as the highest priority.

Medium- and low-severity findings were then addressed to strengthen the overall security posture of the application and reduce supporting weaknesses that could aid further exploitation. This included improving authentication error handling, restricting cross-origin behavior, and removing unnecessary system details from error responses. Although these findings were lower in severity, they still contributed to information disclosure and broader attack surface exposure.

Each remediation step followed the same process: identify the affected code, implement the security fix, re-test the original exploit using the same payload or method documented in the penetration test report, and then verify that the related application feature still functioned correctly. This approach ensured that the fixes not only removed the vulnerability but also preserved normal application behavior. At the end of the remediation process, all eight reported findings had been corrected and successfully re-validated.

3. Remediation Details

3.1 Critical and High Severity Fixes

3.1.1 Vulnerability #1: SQL Injection in Search

Severity: Critical

Location: URL: http://localhost:3001/search

Endpoint: GET /api/search

File: backend/app.py lines 207–218; database/db.py lines 187–193

Description

Team Vortex reported that the search feature built an SQL query by concatenating the user-controlled q parameter into a LIKE clause and passed the full string to cursor.execute() without parameterized placeholders. This allowed an attacker to break out of the intended search pattern and inject arbitrary SQL.

The Problem

The vulnerable implementation treated user input as part of the SQL statement itself. Because the search term was inserted directly into the query string, an attacker could change the logic of the WHERE clause and bypass normal filtering.

Before (Vulnerable Code):

```
# File: database/db.py, Line 187
def search_listings(query):
    """VULNERABILITY: SQL Injection - uses string concatenation"""
    conn = get_connection()
    cursor = conn.cursor()
    sql = "SELECT listings.*, users.username as seller_name FROM listings " \
          "JOIN users ON listings.user_id = users.id " \
          "WHERE listings.title LIKE '%" + query + "%' " \
          "OR listings.description LIKE '%" + query + "%' " \
          "OR listings.category LIKE '%" + query + "%'"
    try:
        cursor.execute(sql)
        listings = cursor.fetchall()
        conn.close()
        return [dict(l) for l in listings]
    except Exception as e:
        conn.close()
        raise e
```

After (Secure Code):

```
# File: database/db.py, Line 187
def search_listings(query):
    """SECURE: Parameterized query prevents SQL injection"""
    conn = get_connection()
    cursor = conn.cursor()
    search_term = f"%{query}%"
    sql = """
        SELECT listings.*, users.username as seller_name
        FROM listings
        JOIN users ON listings.user_id = users.id
        WHERE listings.title LIKE ?
          OR listings.description LIKE ?
          OR listings.category LIKE ?
    """
    try:
        cursor.execute(sql, (search_term, search_term, search_term))
        listings = cursor.fetchall()
        conn.close()
        return [dict(l) for l in listings]
    except Exception as e:
        conn.close()
        raise e
```

What Changed

The vulnerable string concatenation was replaced with parameterized LIKE clauses. The search term is now passed separately as data instead of being inserted into the SQL string. This ensures the database treats the input as text rather than executable SQL.

Security Control Applied

Parameterized queries (prepared statements), CWE-89.

Testing verification

The original proof-of-concept payload ' OR '1'='1 previously returned all 12 listings by bypassing the intended filter logic. After remediation, the same request returned an empty result set. Additional payloads such as ' UNION SELECT NULL-- and '; DROP TABLE users;-- also failed to alter query behavior. A normal search for the textbook still returned valid results, confirming that functionality was preserved.

3.1.2 Vulnerability #2: Cross-Site Scripting (Stored & Reflected XSS)

Severity: Critical

Location(s): Stored XSS: POST /api/listings, Reflected XSS: GET /search

Files:frontend/app/listings/page.jsx:86,frontend/app/listings/[id]/page.jsx:112,120,frontend/app/search/page.jsx:141,163

Description

Team Vortex reported that the application used React's dangerouslySetInnerHTML to render listing titles, descriptions, and reflected search query content without sanitization. This allowed an attacker to inject HTML and JavaScript that executed in users' browsers. Stored payloads affected every visitor who viewed the compromised content, while reflected payloads executed when a user visited a crafted search URL.

The Problem

The frontend bypassed React's built-in escaping by using dangerouslySetInnerHTML on untrusted content. This created both stored and reflected XSS exposure.

Before (Vulnerable Code):

```
// Example instances
<h3 dangerouslySetInnerHTML={{ __html: item.title }} />
<div className="detail-description"
  dangerouslySetInnerHTML={{ __html: listing.description }}
/>
<p dangerouslySetInnerHTML={{ __html: `Showing results for:
<strong>${query}</strong>` }}
/>
```

After (Secure Code):

```
<h3>{item.title}</h3>
<div className="detail-description">{listing.description}</div>
<p>Showing results for: <strong>{query}</strong></p>
```

What Changed

All uses of dangerouslySetInnerHTML tied to user-controlled content were removed and replaced with normal React text rendering. This restored React's default escaping behavior and prevented attacker-supplied HTML or JavaScript from executing in the browser.

Security Control Applied

Output encoding through React auto-escaping, CWE-79.

Testing verification

The original stored XSS payload in a listing title previously executed when the listing page rendered. After remediation, the same payload appeared only as plain text. The reflected XSS payload entered through the search feature also displayed as text rather than executing. A code review confirmed that the affected frontend files no longer used dangerouslySetInnerHTML for those rendering paths.

3.1.3 Vulnerability #3: IDOR with Plaintext Password Exposure

Severity: High

Location(s): http://localhost:3001/profile/[id]

Endpoint: GET /api/profile/<int:user_id>

File: backend/app.py lines 104–119

Description

Team Vortex reported that the profile API required no authentication or authorization and allowed any user or unauthenticated visitor to access any profile by changing the numeric ID in the URL. The API response also included the full user object, including plaintext password, email, and role information. In addition, the application stored passwords in plaintext and returned them in related API flows.

The Problem

This finding combined an access control issue with unsafe credential handling. The application exposed profile data too broadly and treated passwords as normal text throughout storage, authentication, and response serialization.

Before (Vulnerable Code):

```
# File: database/db.py — add_user() stores plaintext
def add_user(username, email, password, role='user'):
    try:
        conn = get_connection()
        cursor = conn.cursor()
        cursor.execute(
            "INSERT INTO users (username, email, password, role) VALUES (?, ?, ?, ?)",
            (username, email, password, role)  # password stored as-is
        )

# File: backend/app.py — login() plaintext compare and password leak
@app.route('/api/login', methods=['POST'])
def login():
    data = request.get_json()
```

```
    username = data.get('username', '')
    password = data.get('password', '')
    user = get_user_by_username(username)
    if user and user['password'] == password:
        return jsonify({'user': user, 'token': f'mock-token-{user["id"]}'})
    if user:
        return jsonify({'error': 'Invalid password'}), 401
    return jsonify({'error': 'User not found'}), 401

# File: backend/app.py — register() returns full user object
user_id = add_user(username, email, password)
if user_id:
    user = get_user_by_id(user_id)
    return jsonify({'user': user, 'token': f'mock-token-{user_id}'}), 201

# File: database/seed.py — plaintext seed data
users = [
    ('admin', 'admin@university.edu', 'admin123', 'admin'),
    ('jsmith', 'john.smith@university.edu', 'password123', 'user'),
    ('emilyr', 'emily.r@university.edu', 'password123', 'user'),
]
```

After (Secure Code):

```
# File: database/db.py — bcrypt import, helper, and hashed storage
import bcrypt

def verify_password(plain_password, hashed_password):
    return bcrypt.checkpw(
        plain_password.encode('utf-8'),
        hashed_password.encode('utf-8')
    )

def add_user(username, email, password, role='user'):
    hashed = bcrypt.hashpw(password.encode('utf-8'), bcrypt.gensalt()).decode('utf-8')
    cursor.execute(
        "INSERT INTO users (username, email, password, role) VALUES (?, ?, ?, ?)",
        (username, email, hashed, role)
    )
```

```
# File: database/__init__.py — verify_password exported
from .db import (..., verify_password)

# File: backend/app.py — login() uses bcrypt and strips password
user = get_user_by_username(username)
if user and verify_password(password, user['password']):
    user_data = {k: v for k, v in user.items() if k != 'password'}
    return jsonify({'user': user_data, 'token': f'mock-token-{user["id"]}'})
return jsonify({'error': 'Invalid username or password'}), 401

# File: backend/app.py — register() strips password
user_id = add_user(username, email, password)
if user_id:
    user = get_user_by_id(user_id)
    user_data = {k: v for k, v in user.items() if k != 'password'}
    return jsonify({'user': user_data, 'token': f'mock-token-{user_id}'}), 201

# File: database/seed.py — hashed seed passwords
def hash_pw(pw):
    return bcrypt.hashpw(pw.encode('utf-8'), bcrypt.gensalt()).decode('utf-8')
users = [
    ('admin', 'admin@university.edu', hash_pw('admin123'), 'admin'),
    ('jsmith', 'john.smith@university.edu', hash_pw('password123'), 'user'),
]

# File: backend/requirements.txt
bcrypt==4.1.2
```

What Changed

Authentication and authorization were enforced for profile access, plaintext password storage was replaced with bcrypt hashing, plaintext comparison in login was removed, and password fields were stripped from login and registration responses. Seeded user accounts were also updated so that passwords were no longer inserted in readable form.

Security Control Applied

Authentication and authorization enforcement, bcrypt password hashing, and sensitive data exclusion from API responses, CWE-639 and CWE-256.

Testing verification

Unauthenticated access to /api/profile/1 previously returned full user data, including plaintext password. After remediation, the same request returned 401 Authentication required. Accessing another user's profile while authenticated returned 403 Access denied. Login and registration responses no longer exposed password values, and database inspection confirmed that stored passwords were now bcrypt hashes instead of plaintext.

3.1.4 Vulnerability #4: Default Credentials in HTML Source

Severity: High

Location(s): http://localhost:3001/login (page source)

File: frontend/app/login/page.jsx lines 126–127

Description

Team Vortex reported that the login page rendered an HTML comment through dangerouslySetInnerHTML that exposed the backend API endpoint and default administrator credentials (admin / admin123). This information was visible in page source and browser developer tools.

The Problem

The application exposed sensitive operational information directly in client-side source. Anyone inspecting the page could recover default credentials and use them to authenticate.

Before (Vulnerable Code):

```
<div dangerouslySetInnerHTML={{ __html:
  "<!-- Debug: API endpoint = .../api/login, Default credentials: admin/admin123 -->" }} />
```

After (Secure Code):

The debug block was deleted entirely.

What Changed

The client-visible debug comment containing credential and endpoint information was deleted from the login page.

Security Control Applied

Removal of sensitive information from client-side source, CWE-200.

Testing verification

After remediation, page source and frontend file review no longer showed the debug credentials comment or the exposed default login string.

3.1.5 Vulnerability #5: Unrestricted File Upload

Severity: High

Location(s): http://localhost:3001/create-listing

Endpoint: POST /api/listings (multipart form)

File: backend/app.py lines 164–167

Description

Team Vortex reported that the listing creation endpoint accepted file uploads with no validation of file type, extension, content, or size. Uploaded files were saved directly to the uploads directory using their original filename and were accessible immediately after upload.

The Problem

Because the upload flow did not validate what was being submitted, a user could upload arbitrary file types and preserve the original filename. This created unnecessary risk from dangerous file content and unsafe file handling.

Before (Vulnerable Code):

```
if 'image' in request.files:
    file = request.files['image']
    if file.filename:
        filepath = os.path.join(UPLOAD_FOLDER, file.filename)
        file.save(filepath)
        image_url = f'/uploads/{file.filename}'

<input type="file" onChange={e => setFile(e.target.files[0])} />
```

After (Secure Code):

```
ALLOWED_EXTENSIONS = {'.jpg', '.jpeg', '.png', '.gif', '.webp'}
MAX_FILE_SIZE = 5 * 1024 * 1024
ext = os.path.splitext(secure_filename(file.filename))[1].lower()
if ext not in ALLOWED_EXTENSIONS:
    return jsonify({'error': 'Invalid file type. Allowed: jpg, jpeg, png, gif, webp'}), 400
safe_name = f"{uuid.uuid4().hex}{ext}"
filepath = os.path.join(UPLOAD_FOLDER, safe_name)

<input type="file" accept="image/jpeg,image/png,image/gif,image/webp" ... />
```

What Changed

The upload process now limits accepted files to approved image extensions, applies a size limit, sanitizes the filename, and saves uploads under randomized UUID-based names instead of the original filename.

Security Control Applied

File validation, filename sanitization, and randomized safe naming, CWE-434.

Testing verification

A malicious upload such as shell.php was previously accepted and saved directly under its original name. After remediation, the same upload was rejected with an invalid file type error. Valid image uploads still succeeded and were stored under UUID-based filenames.

3.2 Medium Severity Fixes

3.2.1 Vulnerability #6: Username Enumeration via Login Errors

Severity: Medium

Location: URL: http://localhost:3001/login

Endpoint: POST /api/login

File: backend/app.py lines 68–71

Description

Team Vortex reported that the login API returned different error messages depending on whether a username existed. It returned Invalid password for an existing account and User not found for a non-existent account. This allowed attackers to enumerate valid usernames.

Before (Vulnerable Code):

```
if user:
    return jsonify({'error': 'Invalid password'}), 401
return jsonify({'error': 'User not found'}), 401
```

After (Secure Code):

```
return jsonify({'error': 'Invalid username or password'}), 401
```

What Changed

The separate authentication failure paths were replaced with one generic error message so that the response no longer reveals whether the account exists.

Security Control Applied

Generic authentication failure messaging, CWE-203 and CWE-204.

Testing verification

After remediation, both an incorrect password for a valid account and a login attempt for a non-existent account returned the same response. This removed the information leak previously used for username enumeration.

3.2.2 Vulnerability #7: Missing CSRF Protection

Severity: Medium

Location(s): POST /api/listings, POST /api/messages, POST /api/register

File: backend/app.py (all POST routes)

Description

Team Vortex reported that state-changing POST endpoints did not implement CSRF token validation and that the backend used CORS(app) with no origin restrictions, allowing requests from any domain.

Before (Vulnerable Code):

```
CORS(app)
```

After (Secure Code):

```
CORS(app, origins=['http://localhost:3000', 'http://localhost:3001'])
```

What Changed

Cross-origin access was restricted to approved frontend origins, and protected operations continued to require authentication tokens. This reduced exposure to unauthorized cross-origin requests.

Security Control Applied

Restricted CORS configuration and authenticated protected operations, CWE-352.

Testing verification

Requests from unapproved origins no longer received a valid Access-Control-Allow-Origin response header, while approved frontend origins continued to function correctly. Protected POST requests without valid authentication were also rejected.

3.3 Low Severity Fixes

3.3.1 Vulnerability #8: Verbose Error Messages

Severity: Low

Location(s): URL: http://localhost:3001/search

Endpoint: GET /api/search

File: backend/app.py lines 215–216

Description

Team Vortex reported that when malformed input caused query errors, the backend returned raw SQLite exception text to the client. This exposed database engine details and query syntax information that could aid further exploitation.

The Problem

The backend returned raw exception text to the client, exposing internal system details and stack traces.

Before (Vulnerable Code):

```
except Exception as e:
    return jsonify({'error': str(e)}), 500
```

After (Secure Code):

```
except Exception as e:
    app.logger.error(f'Search error: {e}')
    return jsonify({'error': 'Search failed. Please try again.'}), 500
```

What Changed

Detailed exception text was removed from client-facing responses and replaced with generic messages, while the full error details were retained in server-side logs. Debug mode was also disabled to reduce exposure of internal application details.

Security Control Applied

Generic client error handling with server-side logging, CWE-209.

Testing verification

Requests that previously returned raw database error text no longer exposed internal details. After remediation, client responses were limited to generic messages, and internal exception information remained only in server-side logs.

4. Testing and Verification

4.1 Testing Methodology

Verification was performed by re-testing each vulnerability with the same payloads, request patterns, or reproduction steps documented in Team Vortex's penetration test report. For each finding, we compared the vulnerable behavior observed before remediation with the behavior of the fixed application after the security changes were applied. In addition to confirming that the original exploit no longer worked, we also tested the related application feature to ensure that normal functionality was preserved. This approach allowed us to validate both the effectiveness of the remediation and the stability of the application after the fixes were introduced.

4.2 Remediation Verification Results

Vulnerability	Exploit Payload	Before (Vulnerable)	After (Fixed)
SQL Injection in Search	' OR '1'='1 in search	HTTP 200; all 12 listings were returned	HTTP 200; {"listings":[]}
Cross-Site Scripting (Stored XSS)	<img src=x onerror=alert('XSS') >	Payload executed in the browser	Payload rendered as plain text
Cross-Site Scripting (Reflected XSS)	<script>alert(1)</script>	Script executed in the browser	Payload displayed safely as text
IDOR with Plaintext Password Exposure	GET /api/profile/1 without authentication	HTTP 200; user data and plaintext password returned	HTTP 401; authentication required
Password Exposure in Login Response	POST /api/login	Password field included in JSON response	Password field removed from response
Plaintext Password Storage	Database inspection	Passwords stored as plaintext values such as admin123 and password123	Passwords stored as bcrypt hashes beginning with $2b$
Default Credentials in HTML Source	View source on /login	admin/admin123 visible in HTML comment	No credentials present in source
Unrestricted File Upload	Upload shell.php	HTTP 201; file saved as shell.php	HTTP 400; invalid file type rejected
Unsafe File Naming	Upload valid_test.jpg	File saved as original filename	File saved as randomized UUID filename

Vulnerability	Exploit Payload	Before (Vulnerable)	After (Fixed)
Username Enumeration via Login Errors	Wrong password for valid user	Invalid password	Invalid username or password
Username Enumeration via Login Errors	Non-existent user	User not found	Invalid username or password
Missing CSRF Protection	Origin: http://evil.com	Access-Control-Allow-Origin header present	No Access-Control-Allow-Origin header returned
Information Disclosure via Verbose Error Messages	Malformed JSON to login	Detailed internal exception output returned	Generic error message returned
Debug Mode Exposure	Flask configuration	debug=True	debug=False

The results show that the original exploit paths identified during the penetration test were no longer effective after remediation. In each case, the vulnerable behavior observed in the initial assessment was replaced with either safe rejection, sanitized output, restricted access, or generic error handling. This confirmed that the implemented fixes successfully addressed the reported findings.

4.3 Functionality Testing

After security remediation was completed, the core features of the application were tested again to confirm that the fixes did not break expected behavior.

Feature	Test	Result
Browse Listings	GET /api/listings	14 listings returned successfully
Category Filter	GET /api/listings?category=electronics	4 results returned successfully
View Listing	GET /api/listings/1	Correct listing data returned
Login (admin)	POST /api/login with admin/admin123	Token returned successfully
Login (jsmith)	POST /api/login with jsmith/password123	Token returned successfully

Feature	Test	Result
Register	POST /api/register with new user data	HTTP 201; token returned successfully
Search	GET /api/search?q=calculus	1 result returned successfully
Search	GET /api/search?q=desk	2 results returned successfully
Send Message	POST /api/messages with valid authentication	HTTP 201; message sent successfully
Upload Image	POST /api/listings with valid .jpg image	HTTP 201; file saved with UUID filename
Profile Access	GET /api/profile/1 with correct token	User data returned successfully

Functionality testing confirmed that the remediation did not disrupt normal application behavior. Core features including listing access, search, authentication, registration, messaging, image upload, and profile access continued to operate correctly after the security changes were applied.

5. Lessons Learned

Technical Lessons

This remediation process reinforced that effective secure coding depends on applying the correct control to the specific vulnerability being addressed. Parameterized queries are the proper solution for SQL injection because they separate code from data. Safe React text rendering is the appropriate fix for the XSS issues identified in this application because it restores built-in output escaping. The project also confirmed that passwords must be protected before storage through strong hashing and that sensitive fields should never be included in API responses.

Process Lessons

The penetration test report significantly improved the remediation process because it already identified the affected files, endpoints, payloads, and recommended fixes. This reduced time spent locating vulnerable code and allowed the team to focus on implementing and verifying corrections. Testing each fix immediately after implementation also proved valuable because it helped identify mistakes early and made it easier to confirm that each vulnerability had actually been resolved.

Future improvements

Future versions of the project should include additional protections beyond the fixes completed in this milestone. Planned improvements include rate limiting on authentication endpoints, use of an ORM such as SQLAlchemy to reduce SQL handling risk, implementation of a Content Security Policy, and integration of security linting and static analysis tools into the development process.

Best Practices Adopted

As a result of this remediation effort, several secure development practices will be carried forward in future work. These include always using parameterized queries for database access, avoiding unnecessary bypasses of framework security protections, validating input at system boundaries, hashing passwords with bcrypt before storage, excluding sensitive data from API responses, keeping client-facing error messages generic, restricting cross-origin access to approved origins, and disabling debug mode outside development use.

6. Conclusion

This report documented the remediation completed for all eight vulnerabilities identified in Team Vortex's Milestone 2 penetration test of the EagleMart application. Each finding was reviewed against the original report, corrected in the affected code, and then re-tested using the same exploit logic or reproduction steps that were used during the assessment. This process confirmed that the reported weaknesses were no longer exploitable after remediation.

The completed fixes addressed critical issues in database security, frontend rendering, access control, password protection, file upload handling, authentication responses, cross-origin behavior, and error handling. Together, these changes significantly improved the overall security posture of the application and reduced the risk of unauthorized access, malicious script execution, sensitive data exposure, and information leakage. Just as important, verification testing confirmed that the application continued to function correctly after the fixes were applied.

Overall, the remediation phase showed that the vulnerabilities identified in the penetration test could be corrected through targeted code changes and more secure development practices. EagleMart now reflects a stronger baseline for secure coding and provides a more defensible implementation of its core features. This milestone also reinforced the importance of reviewing security issues systematically, validating every fix through re-testing, and building security controls directly into the development process rather than adding them only after vulnerabilities are discovered.

Appendix A. Files Modified

The following table summarizes the primary files updated during remediation and the purpose of each change. These modifications correspond directly to the vulnerabilities identified in Team Vortex's penetration test report.

File	Summary of Change
backend/requirements.txt	Added the bcrypt dependency required for secure password hashing
backend/app.py	Updated authentication, registration, CORS configuration, file upload validation, error handling, and debug settings
database/db.py	Replaced vulnerable SQL construction in search and added password verification support
database/__init__.py	Exported the verify_password helper for use by the application
database/seed.py	Replaced plaintext seeded passwords with bcrypt-hashed values
frontend/app/page.jsx	Removed unsafe HTML rendering from listing titles
frontend/app/listings/page.jsx	Removed unsafe HTML rendering from listing titles
frontend/app/listings/[id]/page.jsx	Removed unsafe HTML rendering from listing title and description fields
frontend/app/search/page.jsx	Removed unsafe reflected and stored HTML rendering paths
frontend/app/login/page.jsx	Removed the debug block that exposed default credentials in page source
frontend/app/create-listing/page.jsx	Restricted accepted upload file types in the file input